

# Task 16: Debugging with GDB (GNU Debugger)

Objective:

The objective of this task is to understand how to debug a C++ program using GDB. A program containing a logical error is analyzed by setting breakpoints, inspecting variable values, stepping through execution, identifying the bug, and fixing it.

## 1. Introduction to GDB

GDB (GNU Debugger) is a powerful debugging tool used to analyze programs during execution. It allows developers to pause execution, inspect variables, step through code line by line, and identify logical or runtime errors.

## 2. Problem Description

A C++ program is written to calculate the sum of numbers from 1 to 5. However, the program contains a logical error in the loop condition, resulting in an incorrect output.

## 3. Buggy C++ Program

```
#include <iostream>
using namespace std;

int main() {
    int sum = 0;

    // Logical error: loop condition is incorrect
    for (int i = 1; i < 5; i++) {
        sum += i;
    }

    cout << "Sum = " << sum << endl;
    return 0;
}
```

## 4. Compiling with Debug Symbols

To enable debugging, the program must be compiled with the `-g` flag, which includes debug symbols in the executable.

Command:

```
g++ -g debug_example.cpp -o debug_example
```

## 5. Observing the Incorrect Output

When the program is executed normally, it produces an incorrect result due to the logical error in the loop condition.

Output:

```
Sum = 10
```

## 6. Starting GDB

GDB is started by passing the executable file as an argument.

Command:

```
gdb ./debug_example
```

## 7. Setting Breakpoints

A breakpoint is set at the beginning of the main function to pause execution and inspect program behavior.

Commands:

```
break main
```

```
run
```

## 8. Stepping Through the Program

The program is stepped through line by line using the 'next' command. Variable values are inspected using the 'print' command.

Commands:

```
next
```

```
print i
```

```
print sum
```

## 9. Identifying the Logical Error

By stepping through the loop, it is observed that the loop terminates when *i* reaches 4. The value 5 is never included in the calculation. This indicates an incorrect loop condition.

## 10. Fixing the Error

The loop condition is corrected to include the value 5 by changing the condition from '*i* < 5' to '*i* <= 5'.

Corrected Code:

```
for (int i = 1; i <= 5; i++) {  
    sum += i;  
}
```

## 11. Recompiling and Verifying the Fix

After fixing the code, the program is recompiled and executed again.

Commands:

```
g++ -g debug_example.cpp -o debug_example  
./debug_example
```

Correct Output:

```
Sum = 15
```

## 12. Conclusion

This task demonstrates how GDB can be used to identify and fix logical errors in C++ programs. By setting breakpoints, stepping through execution, and inspecting variables, developers can efficiently debug and validate program logic.